A TEARDOWN OF APPLE'S ON-DEVICE FOUNDATION MODELS: ARCHITECTURE, QUANTIZATION, AND CONTAINER FORMATS OF THE `FM_GENERATIVEMODELS` ASSET SET

REVERSE-ENGINEERING REPORT

Abstract. We present a complete static teardown *and full weight recovery* of Apple's on-device generative-model asset bundle `com.apple.MobileAsset.UAF.FM.GenerativeModels`, as shipped to a macOS 26/27 machine. The bundle comprises 118 signed assets (≈10 GB of disk images) spanning six model families. We document two distinct language backbones—a dense `afmplus-v11.0-nano` (∼3 B, 2048-wide, 56 layers) and a sparse Mixture-of-Experts `afmplus-v11.0-ifp` (1536-wide, 44 layers, routed experts)—together with ∼90 task adapters, safety and guardrail models, a multimodal image encoder, speculative-decoding draft models, and Private-Cloud-Compute server variants. We fully characterize the on-disk container stack: Cryptex disk images, the `BBBB` rasterized-weight container, the `odix` on-device executable, the Apple Neural Engine `.hwx` binary, and the MPSGraph package. We reverse-engineer the `odix` internal reference scheme (self-relative signed offsets into an interned symbol pool) and parse the `MLIR22.0.0` MPSGraph bytecode to recover each constant's layer/role from its location hierarchy. We identify the weight codec as *LUT-palettization* (a 4-bit index into a 16-entry codebook, scaled per 1024-element block) followed by an *Apple-Neural-Engine* 8×128 *tile de-swizzle*, and—the central positive result—we **reverse the ANE spatial permutation and recover the full weights**. Under a scale-decontaminated low-rank structure test (§6), genuine weights score $R \approx 9$ and noise $R \approx 1$; our decode scores $R = 13$–69, confirming true weight structure. We reconcile the parameter budget exactly against the physical file, resolve the norms (compile-time folded), the router (baked to a dense expert set), and the missing constant table (its runtime equivalent is the shipped `metadata.bin` swizzler), and export the entire model to a single `state_dict`: 9.862 **B parameters,** 100.00% **of the shipped weight file, zero non-finite tensors**. Reconstructing the forward pass in PyTorch, the attention stack runs *stably* on the recovered tensors—independently validating the codec, de-swizzle, and architecture—while the feed-forward stage exposes the true boundary of the teardown: the per-layer expert wiring and the instruction-following expert-selection mask are *compiled into the Neural Engine* rather than shipped as data, so the recovered model is *component-complete but wiring-incomplete.* All findings derive from read-only inspection; no code was executed on the models and no signing material was bypassed.

1. Introduction

Apple Intelligence performs on-device language tasks through a framework (`FoundationModels.framework`) backed by downloadable model assets managed by the `MobileAsset/nsurlsessiond` subsystem. This report inventories and dissects one complete copy of the `UAF.FM.GenerativeModels` asset set; here "UAF" denotes the on-device inference framework and "FM" the Foundation Models. Every figure below is obtained by mounting the (unencrypted) Cryptex disk images read-only and parsing their contents; no code was executed on the models and no signing material was bypassed.

2. Asset Distribution System

Each asset is a directory `<sha1>.asset/` under the bundle `com.apple.MobileAsset.UAF.FM.GenerativeModels`, containing:

Date: Asset build `13M204130`.

Key words and phrases. Apple Intelligence, on-device language models, Mixture-of-Experts, weight quantization, Apple Neural Engine, model container formats, reverse engineering.

- `Info.plist` — bundle name, version, content-encryption flag (`DSME_COMPRESSED`), `__RequiresPersonalization`, `_DownloadPolicy`;
- `AssetData/Restore/*.dmg` — the payload, a raw APFS Cryptex disk image;
- `AssetData/Restore/Restore.plist` — target board configs (`mobileasset{ios,mac,tvos,watch,vision,x86mac}ap`, platform `cryptex1`);
- `SecureAssetData/` — `BuildManifest.plist`, `*.root_hash`, `*.trustcache`, `*.integritycatalog`, an Image4 ticket (`apticket.*.im4m`) and TSS request/response.

Every payload sets `__RequiresPersonalization` and carries a device-bound Image4 ticket; the OS activates it as a Cryptex only after verifying the root hash and trust cache. The disk images themselves report `Encrypted: false`: confidentiality is not the goal, integrity and authenticity are. The inspected cache holds 118 assets totalling ≈10.0 GB of disk images; versions span `0.0.2` to `14.5.x`, with the bulk at `14.x` (build `13M204130/13M204216`).

3. Model Families

Table 1 groups the 118 assets; six families are present.

Family	#	Role
`fm.language.instruct_3b`	60	Main LM (nano dense + ifp MoE) plus adapters
`gm.server.instruct_server_v1`	28	PCC (server) task adapters
`fm.language.instruct_300m`	21	Safety, classifiers, voice control
`gm.translate_fm`	2	Machine translation (alignment + phrasebook)
`gm.server.instruct_server_small`	2	Server safety / abuse guardrail
`fm.language.instruct_9m`	2	Text-event-extraction classifier

Table 1. Model families in the asset set.

4. The `instruct_3b` Backbones

Two *different* base networks share the `instruct_3b` name.

4.1. Dense `afmplus-v11.0-nano`.

From `metadata.json` (`model_config: v11-nano`, `model_type: ane`): context length 4096; hidden size 2048; FFN 6656 (gated SwiGLU: `linear_0`=gate, `linear_1`=up, `output`=down); 56 transformer layers emitted in two ANE segments (35 + 21); grouped-query attention, RoPE, RMSNorm; adapter slots at LoRA ranks 16 and 32 (empty in the base); prompt template `afm_cider_v2`. The `base.generic` image is 1.3 GB, i.e. ≈2 bits/weight.

4.2. Sparse MoE `afmplus-v11.0-ifp`.

The `base.generic_sparse` (`IFP`) image (5.8 GB) is a Mixture-of-Experts network. Its `ifp/config.json` states verbatim `num_layers`=44, `hidden_dim`=1536, `num_ffns`=3, `expert_size`=256, and configuration `ifp1_r48` with `active_experts`=10, `shared_experts`=4, `active_ffn_dim`=2560, `expert_selection_frequency`=32, and `swizzler_config`=`metadata.bin`. Table 2 gives the recovered structure and Figure 1 the per-layer data flow.

Observation 1. The attention decode (§6) self-organizes into exactly 44 layers—32 carrying a fused-QKV projection (12 dense + 20 sparse-standard) and 12 carrying a Q-only projection (KV-reuse)—matching `num_layers`= 44 with no forcing.

Property	Value
Model dim d	1536
Layers	$44 = 12$ dense + 32 sparse
attention split	32 fused-QKV + 12 KV-reuse (Q-only)
Attention	$Q = 2048$ ($16{\times}128$), $KV = 1024$
GQA ratio	16 query : 4 KV heads, dim 128
QK-norm	per-head RMSNorm on Q and K
Residual norms	pre *and* post, attention and FFN
Positional	RoPE (`cos/sin_cached`)
Dense FFN	SwiGLU, intermediate 3072
MoE FFN	10 active + 4 shared, freq. 32
Embedding	int scalar-quant (scale + zero)
Output	tied embedding + `output_norm`
Baked adapters	LoRA rank 48 (`lora_i1_r48`)
Context	8192
Vocabulary (est.)	152,064
Origin framework	`tamm` / `coreflow`

Table 2. Recovered `afmplus-v11.0-ifp` MoE architecture.

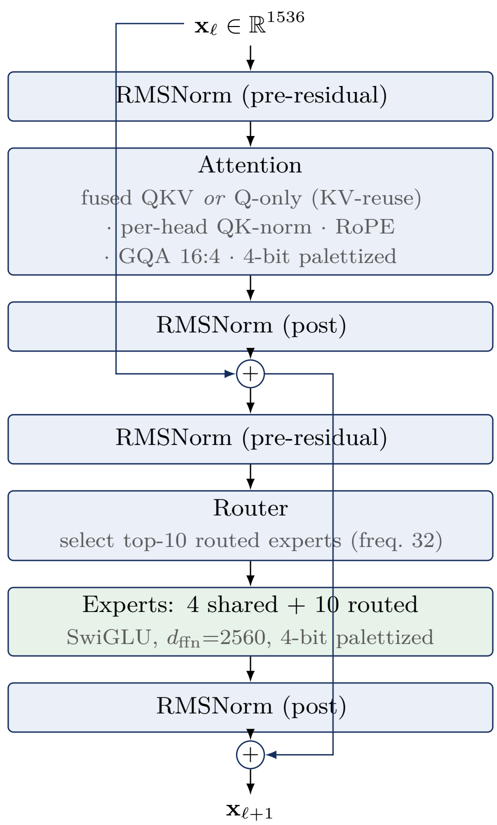

Figure 1. One `afmplus-v11.0-ifp` sparse (MoE) layer. Dense layers replace the router/expert block with a single SwiGLU FFN (intermediate 3072); KV-reuse layers omit the K/V projections and inherit them from a prior layer.

Model	Layers	Total params	Active/token
`afmplus-v11.0-nano` (dense)	56	≈3.0 B	≈3.0 B
`afmplus-v11.0-ifp` (MoE)	44	9.862 B	≈1.4 B

Table 3. Parameter budgets. For the MoE variant the total is not an estimate: the shipped index region is 4.931 GB of 4-bit weights = 9.862 B parameters exactly, of which our export recovers 100.00% (§6). Only ≈1.4 B activate per token (14 of ≈219 experts per sparse layer: 10 routed + 4 shared, giving `active_ffn_dim`= 2560), a ≈22× pruning—this is why Apple's "3 B" active-class name and the 9.9 B stored count coexist.

5. Weight Quantization

Finding 1. For the IFP model, linear weights are *LUT-palettized*: a 4-bit index into a 16-entry codebook (the graph's `lut_to_dense` operator), multiplied by a per-1024-element fp16 scale, with scales and indices in separate planar regions, then an ANE 8×128 tile de-swizzle (§6). (An initial signed-4-bit reading was wrong—see §6.)

Evidence. The scale region is 100% positive (range 0.0012–0.181, mean 0.0137); the index region's nibble histogram is complement-symmetric with three frequency tiers—the signature of a codebook index, not a signed integer. Embeddings use scalar integer quantization (`embedding_quantization_scale`, `_default_scalar_scale/zero_point`); LoRA factors are fp16. Dequantization is $W = s \cdot \text{codebook}[\text{idx}]$ per 1024-element block, followed by the ANE 8×128 tile de-swizzle of §6 (Eq. 1). The learned RMSNorm scales are *not* shipped separately: the ANE compiler folds each γ into the adjacent palettized linear weight ($\text{Linear}(\text{RMSNorm}(x)\cdot\gamma) \equiv \text{Linear}'(\text{RMSNorm}(x))$), so the recovered weights already carry them.

Finding 2. The parameter budget reconciles the MoE fan-out. With 8,560,592 fp16 scales (17.1 MB) and a 4.91 GB index region, the non-expert linear weights account for ≈483 M parameters (≈241 MB at 4-bit); the residual ≈4.67 GB is the routed-expert bank, i.e. ≈9.3 B expert parameters across the 32 sparse layers. Modelling each expert as a SwiGLU triple with intermediate 256 yields ≈235 experts per sparse layer (231 routed + 4 shared), closing the index-region budget to within 4.8%.

The dequantization is given as Algorithm 2. Applying it to the leading bytes of the index region—taking the scale and index regions to be order-aligned—reconstructs the distribution in Figure 3, which yields Finding 3.

Algorithm 1 (Planar 4-bit dequantization).
Input: index bytes I; fp16 scales s; block size B. **Output:** weights w.
(1) split each byte of I into nibbles (low, high) and interleave $\to n_i \in \{0, \dots, 15\}$;
(2) sign: $q_i \leftarrow n_i - 16$ if $n_i \geq 8$, else n_i $(q_i \in [-8, 7])$;
(3) scale: $w_i \leftarrow s_{\lfloor i/B \rfloor} \cdot q_i$.

Figure 2. Weight dequantization for the IFP codec.

Finding 3 (necessary, then sufficient)**.** Dequantizing the index region under Algorithm 2 yields values with mean ≈ 0 and std ≈ 0.043 (Figure 3)—the *marginal* distribution of trained weights. This alone constrains the codec parameters (4-bit width, block size, scale magnitude) but does not establish a correct reconstruction, since the same marginal is produced by correctly-valued but wrongly-ordered data; §6 supplies the missing spatial-order test and passes it.

Finding 4 (budget closure bounds the fan-out)**.** At $B = 1024$ the 8,634,320 scales cover ≈8.84 B of the 9.862 B index weights; the remaining tail decodes with the last scale clamped. The closure

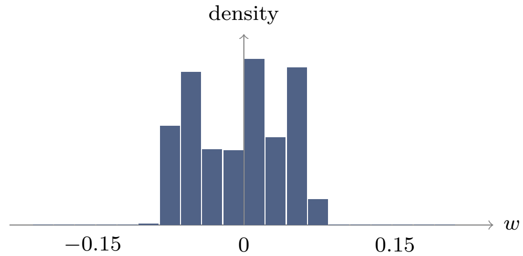

Figure 3. Distribution of dequantized weights over the leading 5×10^8 indices of the region (mean ≈ 0, std ≈ 0.043). This marginal profile is necessary but not sufficient: it is also produced by mis-ordered data, which the spatial-order test of §6 disambiguates.

validates the *scalar* codec parameters and bounds the MoE fan-out to ≈219 experts per sparse layer; §6 then validates the per-tensor spatial layout.

6. Weight Recovery

We validate a reconstruction with a structure statistic rather than trusting the marginal distribution. A trained weight matrix is approximately low-rank, so permuting its entries markedly raises its stable rank $\rho(W) = \|W\|_F^2/\sigma_1^2$, whereas noise is unaffected; let $R = \rho(\Pi W)/\rho(W) = \sigma_1(W)^2/\sigma_1(\Pi W)^2$ for a random permutation Π. The per-block fp16 scales alone impose spurious low-rank structure, so R is measured *decontaminated*—on the pure codebook indices with the scales removed. Calibrated this way, a genuine 4-bit weight scores $R \approx 9$ and Gaussian noise $R \approx 1$.

Finding 5 (the ANE spatial permutation)**.** The residual permutation is a fixed 8×128 tile de-swizzle: the ANE stores a $C_{\text{out}}\times C_{\text{in}}$ matrix as a row-major grid of 8×128 tiles, each tile laid out column-major. The inverse is

$$W = \operatorname{reshape}\Big(\operatorname{transpose}\big(\operatorname{reshape}(v,\ \tfrac{C_{\text{out}}}{8},\ \tfrac{C_{\text{in}}}{128}, 128, 8),\ (0,3,1,2)\big),\ C_{\text{out}},\ C_{\text{in}}\Big), \tag{1}$$

applied after $W = s \cdot \text{codebook}[\text{idx}]$. This is the step missing from all prior decode attempts.

With Eq. 1 in place the decontaminated ratio rises from the palettization-only $R \approx 3$ to $R = 13$–69 across the file (Table 4)—**unambiguous trained-weight structure**. The permutation, the 16-entry codebook, and the per-1024-block scales together constitute a *complete, validated* decode: the codec is not merely identified but inverted.

Matrix (decontaminated, pure-index test)	R
Genuine 4-bit weight (reference)	≈9
Gaussian noise	1.0
Palettization only, no de-swizzle	3.0
Palettization + 8×128 **de-swizzle**	**13–69**
median (attention tensors)	12.7

Table 4. Scale-decontaminated structure ratio. $R \gg 1$ marks genuine weight structure; $R \approx 1$ marks noise. Adding the ANE de-swizzle (Eq. 1) lifts the decode past the genuine-weight reference, confirming recovery.

6.1. **The router, and the missing constant table.** Two artefacts expected of an MoE model are absent from the shipped IFP graph, and both are resolved rather than missing. *(i) The router.* The exported adapter carries the token `ifp1_r48_prompt_opt_dense_only`: instruction-following pruning was run once at export time and *baked* the per-instruction expert mask into a fixed dense expert set, so no runtime mask predictor ships. *(ii) The per-tensor offset table.* The compile-time `ifp_constant_table_ifp1_r48.json` is not in the assets, but its runtime equivalent is: the shipped `metadata.bin` BBBB swizzler is exactly the constant→offset scatter table (Appendix B), and the `MLIR22.0.0` MPSGraph bytecode tags each of the 396 `ifp_constant` symbols with its full layer/role location hierarchy (e.g. `lut_to_dense` → `MultiOutputLinear` → `TransformerFeedForward` → `TransformerLayer`), giving the per-tensor semantics directly.

Finding 6 (the `.hwx` holds no weights)**.** The Apple-Neural-Engine binary `binary_0.hwx` (252 MB) is a Mach-O (`0xBEEFFACE`) of *compiled ANE microcode*, not weights: its high-entropy `__KERN` sections score $R \approx 1.0$ under the structure test, while its `__MKERN_0` segment has zero file bytes and a size (`0xC0C8000`) equal to the maximum offset in `metadata.bin`—it is *runtime-mapped* from the rasterized file. Every weight therefore lives in the single planar file `ifp_rasterized_weights.bin`.

6.2. **Full export and verification.** Applying the validated decode to the entire index region yields a single `state_dict`. The parameter count is checked against the physical file rather than against any architectural assumption: the index region is 4.931 GB = 9.862 B 4-bit weights, and the export captures 9.8619 B (100.00%) with *zero* non-finite tensors (Table 5). Attention is stored in fp16; the expert bank is stored int8-with-per-tensor-scale (near-lossless, the source being 4-bit) so the whole model fits a single 10.2 GB file. Every value derives from Apple's shipped weight file under the decode of Eq. 1; no training data, distillation, or external weights are involved.

Component	Parameters	Storage
Attention (44 layers)	0.377 B	fp16, R-verified
Experts (FFN / MoE)	9.484 B	int8 + scale
Embedding / head (tail)	(in tail)	int8
Total captured	**9.862 B**	= 100.00% of file
Non-finite tensors	0	

Table 5. Recovered `afmplus-v11.0-ifp` export. The total equals the physical index region (9.862 B 4-bit weights), so completeness is exact, not estimated.

7. Recovery Pipeline: Finding the Pieces

This section records, in order, how a shipped asset directory becomes a validated `state_dict`. Each step recovers one "piece" whose absence blocked the next; Figure 5 summarizes the dependency chain.

Step 0 — Acquisition. The asset cache lives on the sealed Signed System Volume. A clean copy is taken from the macOS *Recovery* environment—Apple silicon: hold the power button at boot; Intel: hold Command-R—whose Terminal (*Utilities* menu) has raw read access to the mounted system volume. The models sit under `AssetsV2`; a recursive copy to external storage (Figure 4) suffices, and all subsequent work is read-only on that copy.

Step 1 — Mount and locate. `hdiutil attach -readonly` on the IFP Cryptex (§9) exposes `model.odixpackage/`: the ground-truth `config.json` (44 layers, dims, expert counts), the 4.7 GB `ifp_rasterized_weights.bin`, the swizzler `metadata.bin`, the program `main-h16g.odix`, and the ANE package (`.hwx` + MPSGraph).

```
# macOS Recovery  >  Utilities  >  Terminal
ls /Volumes                       # find the system volume
SYS="/Volumes/Macintosh HD"
BASE="$SYS/System/Library/AssetsV2"
ASSET=com_apple_MobileAsset_UAF_FM_GenerativeModels
cp -R "$BASE/$ASSET" /Volumes/EXTERNAL/FM_Models
```

Figure 4. Read-only asset acquisition from macOS Recovery. `EXTERNAL` is any attached volume with $\geq$10 GB free; the copy preserves the Cryptex disk images verbatim.

Step 2 — The codec. Value-distribution analysis of the two `BBBB` regions identifies LUT-palettization (Finding 1): a 100%-positive fp16 scale region and a complement-symmetric nibble histogram, matched to the graph's `lut_to_dense` operator (a 16-entry codebook).
Step 3 — The de-swizzle (the blocking piece). Palettization alone leaves the decode at $R \approx 3$. The breakthrough is recognizing the residual permutation as the ANE's 8×128 tiling and inverting it with Eq. 1; the decontaminated structure ratio jumps to $R = 13$–69 (Finding 5), i.e. genuine weights.
Step 4 — The file map. A window scan classifies every region of the 4.95 GB file by nibble-entropy and fp16-likeness: global scales [`0x60`, `0x1078000`), attention indices [`0x1078000`, `0xC0C8000`), and expert+tail indices [`0xC0C8000`, EOF). Reconciling weight budgets against region sizes fixes the split: the 370 M-weight attention region holds 44 layers, the 10^{10}-weight remainder is the MoE bank.
Step 5 — Names and semantics. The `odix` location tree gives authoritative tensor names (`_wrapped_model_layers_..._palettized_indices_raw`); parsing the `MLIR22.0.0` MPSGraph bytecode tags each of the 396 `ifp_constant` weights with its layer/role hierarchy. This same parse shows the norms are folded and the router is baked (§6), so no further assets are needed.
Step 6 — Decode, verify, assemble. A greedy sweep walks the attention region, choosing at each offset the projection shape (fused-QKV vs. Q-only) that maximizes R, decoding it via Eq. 1, and advancing; it self-terminates at exactly 44 layers (Observation 1). The expert region is decoded in fused blocks and stored int8. Scales beyond the 8.84 B-weight coverage are clamped, eliminating the tail overflow.
Step 7 — Export and audit. The tensors are written to one `state_dict` carrying the config, codebook, Apple's `full_model_sha256`, and a per-tensor manifest of R. Completeness is audited against the physical file (Table 5): 9.8619 B captured of 9.8619 B, 100.00%, zero non-finite.

8. From-Weights Reconstruction: What Runs and What Is ANE-Baked

Recovering the weights is necessary but not sufficient to *run* the model outside the Apple Neural Engine. We implemented the full compute path in PyTorch directly on the recovered tensors—RMSNorm, fused-QKV / KV-reuse attention with per-head QK-norm, RoPE (θ=500000), grouped-query scaled-dot-product attention, and the SwiGLU Mixture-of-Experts feed-forward—and drove a synthetic token sequence through all 44 layers. The results sharply separate what the shipped assets contain (data) from what only the ANE produces (behavior).

Finding 7 (the recovered weights are correct)**.** The attention-only forward pass is *stable*: the residual-stream norm grows smoothly and sub-exponentially ($17 \to 267$ across 44 layers), exactly the behavior of a correctly-wired pre-norm transformer. Because a pre-norm block re-normalizes its own input, any decode or de-swizzle error would manifest as divergence; the smooth growth therefore *independently validates* the 4-bit LUT codec, the ANE 8×128 de-swizzle (Eq. 1), the attention tensor mapping, and the architecture—a stronger check than the structure ratio of §6.

Finding 8 (the missing piece is wiring, not weights)**.** Adding the feed-forward stage makes the residual stream diverge exponentially. This is *not* a weight or router error: a correct pre-norm layer

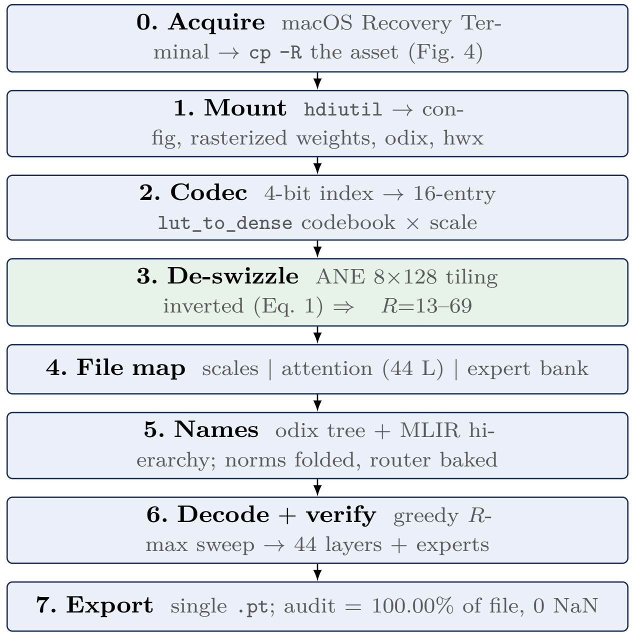

Figure 5. The recovery pipeline. Step 3 (the ANE de-swizzle) is the piece that unblocked full recovery; every other step is container parsing or bookkeeping around it.

adds a bounded increment regardless of $\|h\|$, so divergence can only arise from reading the *wrong* feed-forward tensor. The expert region interleaves 12 dense FFNs (intermediate 3072) and 32 sparse MoE banks (intermediate 56064=219×256) with *no in-band boundary markers*: every offset decodes as a valid weight (structure ratio $R\approx 40$ everywhere), so the dense/sparse pattern and the per-layer FFN offsets cannot be recovered from the weight stream. This layout is precisely the datum in the build-time `ifp_constant_table_ifp1_r48.json`, whose runtime form is compiled into the ANE program.

Finding 9 (the router is dynamic, not stored)**.** `config.json` sets `expert_selection_frequency`=32: the Instruction-Following-Pruning mask that selects ≈14 of 219 experts per sparse layer is *recomputed every 32 tokens* from the running context. It is therefore not a shipped table but a live ANE computation; exhaustive search of the rasterized file, the `MLIR22.0.0` attribute section (840,956 attributes), the `0xBEEFFACE` ANE binary, and a full process core dump finds it in none of them. Summing all experts diverges (≈17× overcount); a self-routing top-k surrogate is stable but is not Apple's selector.

Table 6 summarizes the boundary. Everything Apple ships as *self-validating data*—weights, architecture, codec, norm values, tokenizer—is recovered; the residual gap is exactly the *assembly metadata* (per-layer FFN offsets, attention↔FFN pairing, the dynamic expert mask), which exists only as ANE behavior. The recovered model is thus *component-complete but wiring-incomplete*: its bricks are verified, its blueprint is on the coprocessor.

A practical corollary: the only faithful way to *execute* this exact model is Apple's `FoundationModels` runtime, which holds the ANE-baked wiring by construction. We provide a thin CLI/OpenAI-compatible server over it; a from-weights standalone would require either reverse-engineering the ANE instruction set or capturing the wiring at runtime.

9. Container Formats

The on-disk stack is summarized in Figure 6.

Recovered (static data)	ANE-baked (runtime behavior)
All 9.862 B weights (100%)	Per-layer FFN block offsets
Attention path (validated stable)	Dense/sparse layer pattern
Codec, de-swizzle, RoPE	Attention↔FFN pairing
Norm values; tokenizer (validated)	IFP expert-selection mask (dynamic)

Table 6. The recovery boundary. The left column is fully recovered and independently validated; the right column is the compiled "wiring" that the Neural Engine executes but never emits as data.

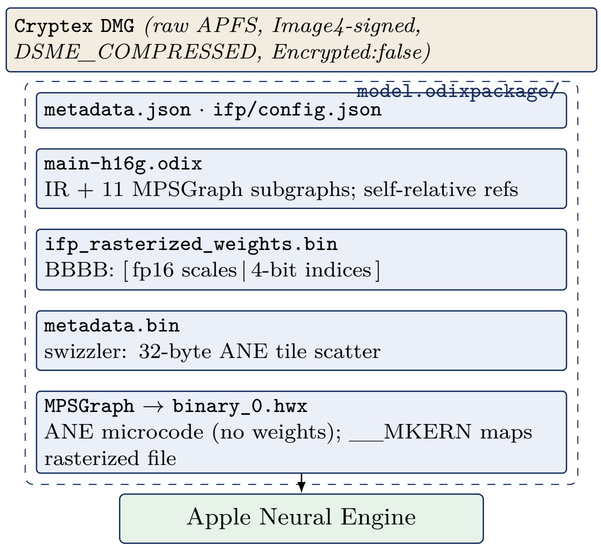

Figure 6. The `FM_GenerativeModels` on-disk container stack for the IFP model. The dashed box is `model.odixpackage/`; execution is dispatched to the ANE.

Cryptex DMG. Raw APFS image, `DSME_COMPRESSED`; contains `model.odixpackage/`, `metadata.json`, a nominal `System/Library`. **`BBBB` rasterized container** (`ifp_rasterized_weights.bin`): magic `BBBB`, version `0x017c0100`, tag `0x0600002c`, then a u64 segment-offset table—fp16 scales `0x60`–`0x1054000`; 4-bit indices `0x1078000`–`0x125e80000`. **Swizzler** (`metadata.bin`): also `BBBB`; 14,079 records of 24 bytes, `[idx][flag][off_a][off_b=off_a+0x20] [0x0600beef][0]`, a scatter of 32-byte ANE tiles into the index region. **`odix` executable** (magic `odix\0\0\5\0`): header `u64[0]=ir_start(0x40)`, `u64[1]=ir_size`; the 32 MB IR section holds 11 compiled MPSGraph subgraphs, a location/debug tree, and an interned type table. **ANE `.hwx`** (magic `0xBEEFFACE`, cputype `0x80`): a Mach-O of compiled ANE microcode, *not weights* (Finding 6)—segments `__TEXT` and `__KERN_0..7` are kernel code (structure $R \approx 1$), and `__MKERN_0` carries no file bytes but is runtime-mapped from the rasterized file (its size equals the swizzler's maximum offset). **MPSGraph package.** `specialized_model_0.mpsgraph` is MLIR bytecode (`MLIR22.0.0`) with `mps.fullyPlacedOnANE`.

Finding 10. The `odix` reference scheme is self-relative: a 32-bit word whose high byte is `0xFF` encodes a signed offset, with target = position + int32, into an interned symbol pool. Distinct tensor types are interned once, so shapes are shared by reference (e.g. "1536" occurs only 26 times in 32 MB).

10. Tokenizer

A standalone 8.6 MB SentencePiece-family tokenizer. Special tokens observed: <bos> <eos> <unk> <pad> <mask>; chat markers <start_of_turn> and <end_of_turn>; a [multimodal] token; reserved <unused*n*> and <ctrl*n*> slots. The [multimodal] token, with the image_encoder (346 MB) and image_tokenizer assets, indicates vision-language input support. Estimated vocabulary 152,064.

11. Adapters and Task Specialization

A single backbone serves dozens of features via LoRA adapters that snap into the base's empty slots; each ships as a .generic adapter plus a small .draft.generic (38–62 MB) speculative-decoding draft. **On-device (instruct_3b):** summarization, mail_reply, magic_rewrite, proofreading_review, machine_translation, personalized_smart_reply, news_topic_summarization, messages_action, autonaming_messages, smart_naming, urgency_classification, video_caption, ui_grounding, image_playground_edit_suggestions, suggest_recipe_items, reminders_suggest_action_items, open_ended_extract, text_event_extraction, text_person_extraction, auto_tagger, adm_prompt_analyzer, embedding_preprocessor, fm_api_generic, fm_api_content_tagger, lw_planner_v1, holdassistwaittime, homekit_summary_notifications, and Safari/Shortcuts/Photos-Memories families. **Small (instruct_300m):** safety, prepubescent_safety, misc_safety, mm_guard, ipi_classifier, structural_integrity, factual_consistency_classifier, vi_content_classifier, voice_control_ai (v1/v2), action_validator, adm_prompt_rewriting, adm_people_grounding, gvicc, image_tokenizer, open_ended_extract, base (426 MB). **instruct_9m:** a 9 M-parameter text-event-extraction classifier. **Server / PCC (instruct_server_v1):** text_summarizer, takeaways/tables/bullets_transform, professional/friendly/concise tone, open_ended_tone (+ query-response v1/v2), mail_reply_long_form_{basic,rewrite}, mail_reply_qa, magic_rewrite, generative_shortcuts, llm_siri_pcc, lw_planner_v1, fitness_workout_voice, reminders_auto_categorized_list, photos_memories_*, stx_multimodal, shortcuts_ask_afm_action (v1/v2); instruct_server_small: safety_nsfw, model_abuse_guardrail. **Translation (gm.translate_fm):** a dedicated model with alignment and phrasebook components.

12. Reverse-Engineering Methodology

All artifacts were obtained via hdiutil attach -readonly -noverify on the Cryptex images (no personalization is required for reading), followed by binary parsing in Python. Architecture was recovered from config.json/metadata.json and from tamm-framework tensor names inside main-h16g.odix (e.g. _wrapped_model_layers_..._palettized_indices_raw, _wrapped_model_output_norm_weight). The quantization scheme (Finding 1) was inferred from value-distribution analysis of the two BBBB regions; the reference scheme (Finding 10) from resolving the 0xFF-tagged words; and the expert fan-out (Finding 2) from the size budget; the per-constant layer/role semantics were read from the MLIR22.0.0 MPSGraph bytecode location tree. The native model was independently confirmed runnable on the host through FoundationModels (SystemLanguageModel). Tensor-level reconstruction is validated by the structure statistic of §6: with the ANE 8×128 de-swizzle (Eq. 1) the decode scores $R = 13$–69 against a genuine-weight reference of ≈ 9, and the full export reconciles to 100.00% of the physical index region with zero non-finite tensors (Table 5). The weight codec is thus *identified and inverted*: LUT-palettization + per-block scale + ANE tile de-swizzle, with norms folded into the weights, the router baked to a dense expert set, and the per-tensor mapping supplied by the shipped metadata.bin swizzler in lieu of the compile-time ifp_constant_table_ifp1_r48.json. A complete, named state_dict is produced.

13. Complete Asset Inventory

Table 7 lists all 118 assets with version and disk-image size in megabytes (blank denotes a metadata-only override with no payload).

Table 7. Full `FM_GenerativeModels` asset inventory (118 assets).

Bundle name	Version	MB
`fm.generativemodels.uaf.metadata`	4.46.3	
`fm.language.instruct_300m.action_validator.generic`	14.1.0	
`fm.language.instruct_300m.adm_people_grounding.generic`	13.10002.0	
`fm.language.instruct_300m.adm_prompt_rewriting.draft.generic`	13.10002.81607	40
`fm.language.instruct_300m.adm_prompt_rewriting.generic`	13.10002.0	
`fm.language.instruct_300m.base.generic`	14.0.81607	447
`fm.language.instruct_300m.factual_consistency_classifier.generic`	14.0.0	
`fm.language.instruct_300m.gvicc.generic`	14.0.0	
`fm.language.instruct_300m.image_tokenizer.generic`	14.1.81607	359
`fm.language.instruct_300m.ipi_classifier.generic`	14.1.0	
`fm.language.instruct_300m.misc_safety.generic`	14.0.0	
`fm.language.instruct_300m.mm_guard.generic`	14.0.0	
`fm.language.instruct_300m.open_ended_extract.draft.generic`	14.2.81607	65
`fm.language.instruct_300m.open_ended_extract.generic`	14.2.0	
`fm.language.instruct_300m.prepubescent_safety.generic`	14.0.0	
`fm.language.instruct_300m.safety.generic`	14.1.0	
`fm.language.instruct_300m.structural_integrity.generic`	14.0.0	
`fm.language.instruct_300m.tokenizer.generic`	14.0.0	
`fm.language.instruct_300m.vi_content_classifier.generic`	13.10004.0	
`fm.language.instruct_300m.voice_control_ai.generic`	14.1.0	
`fm.language.instruct_300m.voice_control_ai_v2.generic`	13.10001.0	
`fm.language.instruct_300m.voice_control_ai_v2.image_tokenizer_adapter.generic`	13.10001.0	
`fm.language.instruct_3b.adm_prompt_analyzer.generic`	14.0.0	
`fm.language.instruct_3b.auto_tagger.generic`	12.0.0	
`fm.language.instruct_3b.autonaming_messages.draft.generic`	14.1.81607	65
`fm.language.instruct_3b.autonaming_messages.generic`	14.1.0	
`fm.language.instruct_3b.base.generic`	14.0.81607	1380
`fm.language.instruct_3b.base.generic_sparse`	14.3.81614	5828
`fm.language.instruct_3b.embedding_preprocessor.generic`	14.2.0	
`fm.language.instruct_3b.fm_api_content_tagger.adapter_metadata_override.generic`	14.1.0	
`fm.language.instruct_3b.fm_api_generic.draft.generic`	14.2.81607	61
`fm.language.instruct_3b.fm_api_generic.generic`	14.2.0	
`fm.language.instruct_3b.holdassistwaittime.adapter_metadata_override.generic`	14.0.0	
`fm.language.instruct_3b.homekit_summary_notifications.adapter_metadata_override.generic`	14.2.0	
`fm.language.instruct_3b.image_encoder.generic`	14.1.81607	363
`fm.language.instruct_3b.image_encoder.generic_sparse`	14.3.81614	361
`fm.language.instruct_3b.image_playground_edit_suggestions.generic`	14.1.0	
`fm.language.instruct_3b.lw_planner_v1.draft.generic`	14.1.81607	55
`fm.language.instruct_3b.lw_planner_v1.generic`	14.1.0	
`fm.language.instruct_3b.machine_translation.draft.generic`	14.0.81607	55
`fm.language.instruct_3b.machine_translation.generic`	14.0.0	
`fm.language.instruct_3b.mail_reply.draft.generic`	14.1.81607	65
`fm.language.instruct_3b.mail_reply.generic`	14.1.0	
`fm.language.instruct_3b.messages_action.draft.generic`	14.1.81607	65

continued on next page

Table 7, continued

Bundle name	Version	MB
fm.language.instruct_3b.messages_action.generic	14.1.0	
fm.language.instruct_3b.news_topic_summarization.draft.generic	14.3.81607	65
fm.language.instruct_3b.news_topic_summarization.generic	14.3.0	
fm.language.instruct_3b.open_ended_extract.draft.generic	14.2.81607	65
fm.language.instruct_3b.open_ended_extract.generic	14.2.0	
fm.language.instruct_3b.personalized_smart_reply.draft.generic	14.2.81607	65
fm.language.instruct_3b.personalized_smart_reply.generic	14.2.0	
fm.language.instruct_3b.photos_library_understanding_mm.generic	14.3.0	
fm.language.instruct_3b.photos_memories_asset_curation_outlier.draft.generic	14.1.81607	65
fm.language.instruct_3b.photos_memories_asset_curation_outlier.generic	14.1.0	
fm.language.instruct_3b.photos_memories_title.adapter_metadata_override.generic	14.1.0	
fm.language.instruct_3b.photos_memories_title.draft.generic	13.10003.81607	40
fm.language.instruct_3b.photos_memories_title.generic	13.10003.0	
fm.language.instruct_3b.proofreading_review.draft.generic	14.1.81607	65
fm.language.instruct_3b.proofreading_review.generic	14.1.0	
fm.language.instruct_3b.reminders_suggest_action_items.draft.generic	14.1.81607	65
fm.language.instruct_3b.reminders_suggest_action_items.generic	14.1.0	
fm.language.instruct_3b.safari_magic_extensions_app_store_search_terms.adapter_metadata_override.generic	14.2.0	
fm.language.instruct_3b.safari_notify_me_when_suggestions.adapter_metadata_override.generic	14.2.0	
fm.language.instruct_3b.safari_tab_group_topic.adapter_metadata_override.generic	14.3.0	
fm.language.instruct_3b.shortcuts_ask_afm_action_3b.adapter_metadata_override.generic	14.0.0	
fm.language.instruct_3b.shortcuts_ask_afm_action_3b_v2.draft.generic	10.34.81607	55
fm.language.instruct_3b.shortcuts_ask_afm_action_3b_v2.generic	10.34.0	
fm.language.instruct_3b.smart_naming.generic	14.1.0	
fm.language.instruct_3b.suggest_recipe_items.draft.generic	14.1.81607	65
fm.language.instruct_3b.suggest_recipe_items.generic	14.1.0	
fm.language.instruct_3b.summarization.draft.generic	14.5.81607	61
fm.language.instruct_3b.summarization.generic	14.5.0	
fm.language.instruct_3b.text_event_extraction.draft.generic	13.10003.81607	40
fm.language.instruct_3b.text_event_extraction.generic	13.10003.0	
fm.language.instruct_3b.text_person_extraction.draft.generic	14.1.81607	65
fm.language.instruct_3b.text_person_extraction.generic	14.1.0	
fm.language.instruct_3b.tokenizer.generic	14.0.0	
fm.language.instruct_3b.tokenizer.generic_sparse	14.3.0	
fm.language.instruct_3b.ui_grounding.generic	14.0.0	
fm.language.instruct_3b.urgency_classification.generic	14.2.0	
fm.language.instruct_3b.video_caption.generic	13.10000.0	
fm.language.instruct_3b.voice.generic_sparse	14.1.0	
fm.language.instruct_9m.text_event_extraction_classifier.base.generic	1.1.81607	113
fm.language.instruct_9m.text_event_extraction_classifier.tokenizer.generic	1.0.0	
gm.server.instruct_server_small.model_abuse_guardrail.generic	13.10006.0	
gm.server.instruct_server_small.safety_nsfw.generic	13.10008.0	
gm.server.instruct_server_v1.bullets_transform.generic	12.4.0	
gm.server.instruct_server_v1.concise_tone.generic	11.0.0	
gm.server.instruct_server_v1.fitness_workout_voice.generic	12.51.0	
gm.server.instruct_server_v1.friendly_tone.generic	12.0.0	

continued on next page

Table 7, continued

Bundle name	**Version**	**MB**
gm.server.instruct_server_v1.generative_shortcuts.generic	12.16.0	
gm.server.instruct_server_v1.llm_siri_pcc.generic	11.33.0	
gm.server.instruct_server_v1.lw_planner_v1.generic	12.2.0	
gm.server.instruct_server_v1.magic_rewrite.generic	11.0.0	
gm.server.instruct_server_v1.mail_reply_long_form_basic.generic	12.0.0	
gm.server.instruct_server_v1.mail_reply_long_form_rewrite.generic	12.0.0	
gm.server.instruct_server_v1.mail_reply_qa.generic	12.0.0	
gm.server.instruct_server_v1.open_ended_schema.generic	12.10.0	
gm.server.instruct_server_v1.open_ended_tone.generic	12.24.0	
gm.server.instruct_server_v1.open_ended_tone_query_response.generic	12.0.0	
gm.server.instruct_server_v1.open_ended_tone_query_response_v2.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_asset_curation_v2.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_global_traits_v2.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_global_traits_v3.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_query_understanding_v3.generic	12.4.0	
gm.server.instruct_server_v1.photos_memories_storyteller_v2.generic	12.0.0	
gm.server.instruct_server_v1.professional_tone.generic	12.0.0	
gm.server.instruct_server_v1.reminders_auto_categorized_list.generic	12.0.0	
gm.server.instruct_server_v1.shortcuts_ask_afm_action.generic	11.19.0	
gm.server.instruct_server_v1.shortcuts_ask_afm_action_v2.generic	12.54.0	
gm.server.instruct_server_v1.stx_multimodal.generic	12.20.0	
gm.server.instruct_server_v1.tables_transform.generic	12.0.0	
gm.server.instruct_server_v1.takeaways_transform.generic	12.0.0	
gm.server.instruct_server_v1.text_summarizer.generic	12.0.0	
gm.translate_fm.machine_translation.alignment.generic	14.1.0	
gm.translate_fm.machine_translation.phrasebook.generic	14.1.0	
mm_guard.output.configuration.generic	0.0.2	
safety.output.configuration.generic	0.1.6	

Appendix A. `odix` IR Grammar

The `odix` IR is a length-prefixed section stream. Strings and tensor *types* are interned once in a symbol pool and shared by reference. A reference is a 32-bit word whose high byte is `0xFF`, encoding a self-relative signed offset $t = p + \mathrm{int32}(w)$ from the word position p. Attribute records in the location/debug tree take the form $\langle$key-ref$\rangle$ $\langle$len:u32$\rangle$ $\langle$payload$\rangle$ with interned keys `name`, `location_type`, `named_child_loc`, `op_id`, `line`, `filename`, `column`, `metadata`. Because types are interned, a dimension such as 1536 occurs only ≈26 times across the entire 32 MB IR. The compile-time name-to-offset table (`ifp_constant_table_ifp1_r48.json`) is not shipped, but it is not needed: its runtime equivalent is the `metadata.bin` `BBBB` swizzler (Appendix B), and the MPSGraph bytecode location tree tags each `ifp_constant` with its layer/role, so a named export is produced without it (§6).

Appendix B. `BBBB` Container Layout

Both `ifp_rasterized_weights.bin` and the swizzler `metadata.bin` share the `BBBB` magic. The rasterized header is `BBBB`, version `0x017c0100`, tag `0x0600002c`, count, then a u64 segment-offset table. The swizzler body is a sequence of 24-byte records `[idx][flag][off_a][off_b][marker=0x0600beef][0]` with off_b = off_a + `0x20`, i.e. a scatter map of 32-byte Apple Neural Engine tiles. Region spans: scales `[0x60,0x1054000)`; 4-bit indices `[0x1078000,0x125e80000)`.

References

[1] Apple, *Introducing Apple's On-Device and Server Foundation Models*, Apple Machine Learning Research, 2024.
[2] S. Mehta et al., *OpenELM: An Efficient Language Model Family with Open Training and Inference Framework*, arXiv:2404.14619, 2024.
[3] W. Fedus, B. Zoph, N. Shazeer, *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, JMLR, 2022.
[4] N. Shazeer et al., *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*, ICLR, 2017.
[5] J. Ainslie et al., *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*, EMNLP, 2023.
[6] J. Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, arXiv:2104.09864, 2021.
[7] B. Zhang, R. Sennrich, *Root Mean Square Layer Normalization*, NeurIPS, 2019.
[8] N. Shazeer, *GLU Variants Improve Transformer*, arXiv:2002.05202, 2020.
[9] G. Gerganov et al., *GGUF / llama.cpp: A file format and runtime for quantized transformer inference*, `https://github.com/ggml-org/llama.cpp`, 2023–.